UNIVERSIDAD NACIONAL DE INGENIERÍA / FACULTAD DE SISTEMAS

CURSO: PRUEBAS Y ASEGURAMIENTO DE LA CALIDAD DE SOFTWARE (QA)

Automatización de Pruebas de Software con PHPUnit 10.5

Auditoría y Certificación del Sistema de Facturación Electrónica SUNAT UBL 2.1

INTEGRANTES DEL EQUIPO:

Apolinario Rodriguez, Luis Fernando • Umeres Vilca, Gustavo Aron

Vasquez Falcón, Mariana Ysabel • Ventocilla Alcantara, Alessandro Tito

Lima, Perú • 2026

1. ¿Qué es PHPUnit y por qué lo usamos?

El Estándar de la Industria

PHPUnit es el framework de pruebas de facto en el ecosistema PHP, desarrollado por Sebastian Bergmann.

- **PHPUnit 10.5:** Diseñado para PHP 8.2+, incorpora tipado estricto sin coerción débil.
- **Cero dependencias externas:** Ejecuta pruebas puras en memoria.
- **Orquestación XML:** Gestionado mediante `phpunit.xml` con control de salida y reporte Testdox.

Arquitectura del Runner

- **Jerarquía de Suites:** Separa pruebas por velocidad y aislamiento (Unit, Integration, Feature).
- **Motor de Aserciones:** Más de 80 aserciones nativas para validar estados y tipos estrictos.
- **Dobles de Prueba Nativos:** Soporte directo para Mocks, Stubs y Spies sin requerir dependencias pesadas.

2. El Ciclo de Vida de las Pruebas (Test Lifecycle)

Para garantizar que cada prueba sea **independiente y aislada**, PHPUnit implementa métodos de enganche (*lifecycle hooks*):

1. **setUpBeforeClass():** Se ejecuta una sola vez antes de la suite (ej. compilar esquemas XSD).
2. **setUp():** Se ejecuta **antes de cada test individual**. En nuestro sistema, reinicia la base de datos SQLite en memoria (`:memory:`).
3. **testMethod():** Ejecuta la lógica del caso de prueba.
4. **tearDown():** Limpia objetos en memoria y cierra conexiones.
5. **tearDownAfterClass():** Liberación final de memoria.

LifecycleHooksTest.php

```php
protected function setUp(): void
{
    parent::setUp();
    // 1. Instancia BD SQLite en RAM
    $this->pdo = new PDO('sqlite::memory:');
    $this->pdo->setAttribute(
        PDO::ATTR_ERRMODE,
        PDO::ERRMODE_EXCEPTION
    );
    // 2. Siembra catalogo inicial
    $this->seeder->run($this->pdo);
}

protected function tearDown(): void
{
    $this->pdo = null; // Destruye RAM
    parent::tearDown();
}
```

3. Atributos de PHP 8 (Decoradores): #[DataProvider]

Pruebas Parametrizadas Elegantes:

En versiones antiguas se usaban anotaciones en comentarios (`@dataProvider`). En PHPUnit 10 usamos los **atributos nativos de PHP 8**:

- **#[DataProvider]:** Alimenta un test con una matriz de datos sin repetir código.
- **#[Test]:** Declara explícitamente un método de prueba sin requerir prefijo `test`.
- **Ventaja en QA:** Permite evaluar matrices de frontera (Valores Límite) con múltiples RUCs en 1 solo método.

ValueObjectsTest.php

```php
#[DataProvider('proveedorRucsValidos')]
public function testAceptaRucsValidos(
    string $ruc,
    string $tipo
): void {
    $vo = new Ruc($ruc);
    $this->assertSame($ruc, $vo->valor());
}

public static function proveedorRucsValidos():
     array
{
    return [
        'SUNAT'    => ['20131312955', 'Empresa'
            ],
        'BCP'      => ['20100047218', 'Empresa'
            ],
        'ALICORP'  => ['20100055237', 'Empresa'
            ],
        'PERSONA'  => ['10456789124', 'Natural'
            ],
    ];
}
```

4. Catálogo de Aserciones: assertSame vs assertEquals

¡Cuidado con la Igualdad Débil!

- **assertEquals($a, $b):** Usa comparación débil (==). En PHP, "100" == 100 da verdadero. En facturación esto puede ocultar bugs de tipado.
- **assertSame($a, $b):** Usa identidad estricta (===). Comprueba que el valor y el tipo de dato sean exactamente iguales (`float` con `float`).

Aserciones Críticas de Excepción

expectException(Clase::class): Comprueba que el sistema rechace operaciones inválidas lanzando excepciones de dominio específicas.

ComprobanteDomainRulesTest.php

```php
// Regla 2017: Factura exige RUC, no DNI
public function testImpideFacturaAClienteConDni(): void
{
    $this->expectException(DomainValidationException::class);
    $this->expectExceptionMessage("Solo se emiten Facturas a clientes con RUC");

    // Act: Intenta emitir factura a DNI 8 digitos
    $emisor = new Emisor(new Ruc('20131312955'));
    $clienteDni = new Cliente(new Dni('72345678'));

    $factura = Comprobante::crearFactura('F001', 1, $emisor, $clienteDni);
}
```

5. Dobles de Prueba: Mocks y Spies de SUNAT

¿Por qué no llamamos al Web Service real de SUNAT?

- Los servidores del Estado sufren caídas de red frecuentes.
- Requieren certificados digitales pagados y conexión a internet.
- Tienen lentitud de 2 a 5 segundos por comprobante.

La Solución de QA: Mocks y Spies nativos

- **Mock:** Simula respuestas programadas del CDR (Aceptado o Rechazado 2800).
- **Spy:** Verifica con `expects($this->never())` que si falla el stock, la pasarela de red **jamás sea invocada**.

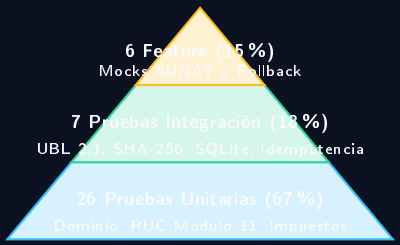
FacturacionMocksFeatureTest.php

```php
// Spy de proteccion: Si no hay stock, NUNCA llama a SUNAT
public function testNoLlamaASunatSiNoHayStock(): void
{
    // 1. Creamos el Mock de SUNAT como Spy
    $sunatSpy = $this->createMock(
        SunatGatewayInterface::class
    );
    // Invariante: expects(never())
    $sunatSpy->expects($this->never())
             ->method('enviarComprobante');

    $this->expectException(
        StockInsuficienteException::class
    );

    // 2. Intenta vender 50 laptops (solo hay 15)
    $service = new FacturacionService(
        $this->kardex, $sunatSpy
    );
    $service->emitirFactura($ordenSinStock)
        ;
```

6. Pirámide de Pruebas de Mike Cohn en Nuestro Proyecto



Métricas Oficiales de Ejecución

- **Total de Pruebas:** 39 casos de prueba.
- **Aserciones Evaluadas:** 140 aserciones estrictas.
- **Fallos / Errores:** 0 (100 % de éxito en verde).
- **Tiempo Total de Ejecución: 0.21 segundos.**
- **Consumo de RAM:** 12 MB (usando SQLite `:memory:`).

7. Caso de Prueba 1: Algoritmo SUNAT Módulo 11 en Value Object Ruc

Fórmula Oficial de Ponderación:

SUNAT valida el RUC aplicando los factores:

$$\vec{F} = [5, 4, 3, 2, 7, 6, 5, 4, 3, 2]$$

- Se multiplica cada uno de los 10 primeros dígitos por su factor.
- Se suma el total: $S = \sum d_i \cdot f_i$.
- Residuo: $R = 11 - (S \pmod{11})$.
- Si $R == 10 \implies 0$; Si $R == 11 \implies 1$.
- El dígito verificador debe coincidir exactamente con el dígito 11.

src/Domain/ValueObjects/Ruc.php

```php
private function validarModulo11(string $ruc): void
{
    $factores = [5, 4, 3, 2, 7, 6, 5, 4, 3, 2];
    $suma = 0;
    for ($i = 0; $i < 10; $i++) {
        $suma += ((int) $ruc[$i]) * $factores[$i];
    }
    $resto = 11 - ($suma % 11);
    $digitoVerificador = match ($resto) {
        10 => 0,
        11 => 1,
        default => $resto
    };
    if ($digitoVerificador !== (int) $ruc[10]) {
        throw new DomainValidationException(
            "El digito verificador del RUC es invalido"
        );
    }
```

8. Caso de Prueba 2: Liquidación Tributaria Mixta Compleja

Regla Tributaria Multirégimen:

En Perú, una misma factura puede incluir:

- **Bienes Gravados (10):** Sujetos al 18 % de IGV.
- **Libros Exonerados (20):** Ley 31053 (0 % IGV).
- **Muestras Gratuitas (15):** IGV referencial que **no suma** al total a pagar.
- **Bolsas Plásticas (ICBPER):** Impuesto fijo de S/ 0.50 por bolsa.

Aserción Clave: Garantizar que las bolsas fiscales no se mezclen ni contaminen el valor de venta neto.

CalculadoraTributariaTest.php

```php
public function testCalculaComprobanteMixto(): void
{
    $calc = new CalculadoraTributaria();
    $totales = $calc->calcular([
        new Item('Monitor', 1000.0, 1, '10'), // Gravado
        new Item('Libro',   200.0,  1, '20'), // Exonerado
        new Item('Muestra', 50.0,   1, '15'), // Gratuito
        new Item('Bolsa',   0.10,   2, '10', icbper: true)
    ]);

    $this->assertSame(1000.0, $totales->gravado);
    $this->assertSame(180.0,  $totales->igv);
    $this->assertSame(200.0,  $totales->exonerado);
    $this->assertSame(1.00,   $totales->icbper); // 2 x 0.50
    $this->assertSame(1381.20, $totales->
```

9. Caso de Prueba 3: Seguridad Criptográfica (Tamper Test)

Ataque de Alteración de Montos:

Simulamos un atacante que intercepta la factura XML ya firmada digitalmente con SHA-256 y cambia el monto de **S/ 1,180.00** a **S/ 10.00**:

1. El verificador criptográfico canónico recalcula el digest del XML alterado.
2. El digest no coincide con el `<ds:DigestValue>` firmado por la llave privada.
3. El simulador de SUNAT rechaza la factura arrojando el **Código de Error Fiscal 2335**.

XmlAndCryptoIntegrationTest.php

```php
public function testDetectaAdulteracionDeImporte(): void
{
    $xmlFirmado = $this->generador->crearXml($factura);

    // ATAQUE: Modifica monto en el XML
    $xmlAdulterado = str_replace(
        '<cbc:PayableAmount currencyID="PEN">1180.00</cbc:PayableAmount>',
        '<cbc:PayableAmount currencyID="PEN">10.00</cbc:PayableAmount>',
        $xmlFirmado
    );

    // Evaluacion del validador de SUNAT
    $resultado = $this->verificador->validar($xmlAdulterado);

    $this->assertFalse($resultado->esValido);
    $this->assertSame(2335, $resultado->
```

10. Caso de Prueba 4: Persistencia e Idempotencia (Doble Clic)

El Problema del Doble Clic del Cajero:

Si el operador de caja pulsa dos veces seguidas el botón Çobrar.° se interrumpe la red:

- Un sistema tradicional emitiría dos facturas distintas cobrando el doble al cliente.
- Nuestro sistema usa una tabla de **Idempotency Keys**:

  UUID Clave $\Longrightarrow$ Estado: IN_PROGRESS $\Longrightarrow$ COMPLE
- La segunda petición detecta la clave y **devuelve la respuesta en caché** sin duplicar el comprobante.

SqlitePersistenceIntegrationTest.php

```php
public function testTablaIdempotenciaPrevieneDobleEmision(): void
{
    $key = 'req-uuid-987654321';

    // Intento 1: Registro exitoso
    $res1 = $this->idempotency->lockAndExecute(
        $key, fn() => $this->emitirFactura($orden)
    );

    // Intento 2: Reintento inmediato con misma clave
    $res2 = $this->idempotency->lockAndExecute(
        $key, fn() => $this->emitirFactura($orden)
    );

    // Ambas respuestas son identicas
    $this->assertSame($res1->correlativo, $res2->correlativo);
```

11. Evidencias del Sistema en Ejecución: Facturador y Kardex

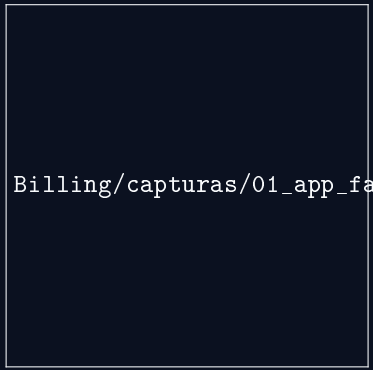


Figura 1: Emisión en tiempo real en `localhost:5173`

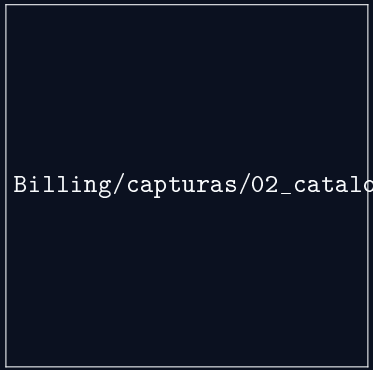


Figura 2: Catálogo de 9 productos y Kardex atómico

12. Evidencias del Sistema en Ejecución: Historial y Visor UBL 2.1

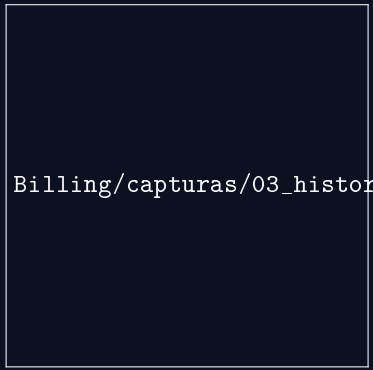


Figura 3: Bandeja fiscal con hash SHA-256

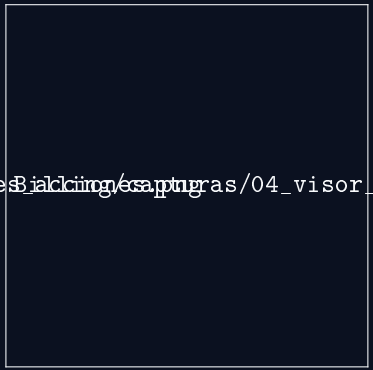


Figura 4: Visor del XML estructurado con firma digital

13. Evidencia Oficial: Salida Íntegra de Terminal PHPUnit 10.5

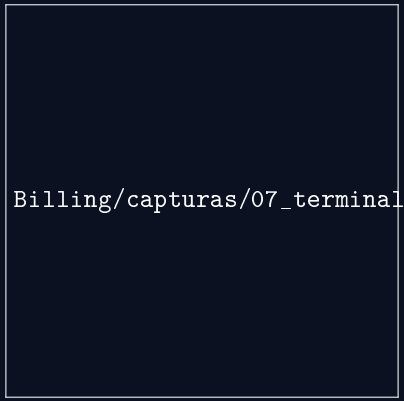


Figura 7: 39 Tests en Verde • 140 Aserciones Auditadas • 0.21 segundos en RAM

14. Conclusiones y Evaluación del Proceso de Pruebas

Aportes de PHPUnit 10 al Proyecto

- **Aislamiento Absoluto:** SQLite `:memory:` permite ejecutar 39 pruebas en 0.21 segundos sin contaminar disco ni bases de datos de producción.
- **Cero Dependencia de Red:** Los Mocks y Spies permiten simular caídas HTTP 504 y rechazos tributarios 2800 de SUNAT de forma 100 % controlada.
- **Blindaje Criptográfico:** Las pruebas de seguridad validan que cualquier intento de adulteración sea detectado inmediatamente antes de enviarse al OSE.

Certificación de Calidad

- **Tasa de Aprobación:** 100 % (39/39 pruebas).
- **Aserciones Cumplidas:** 140/140 verificadas.
- **Defectos Encontrados y Corregidos:** 3 incidencias críticas resueltas (flag `-colors=never`, UTF-8 BOM sanitization y expansión de catálogo a 9 ítems).
- **Estado Final:** APROBADO SATISFACTORIAMENTE.

¡Muchas Gracias por su Atención!

Estamos listos para la ronda de preguntas y sustentación de código

Repositorio Oficial en GitHub:

`https://github.com/fujimor1/Billing.git`

39 Tests Automatizados • PHPUnit 10.5 • SUNAT UBL 2.1